

WebGL 3D Geometry Assignment Writeup

Student Name: Aisha Tursynbay

Student ID Ending: 21

Course: Computer Graphics / WebGL Assignment

Repository: github.com/atursynba/WebGL-3D-Geometry-Assignment

1. Parameter Calculation & Variant Analysis

Based on the assignment formula using the student ID ending in **21**:

1. **Depth Projection Offset (ox,oy):**
 - **Second-to-last digit (2):** $2 \bmod 4 = 2 \Rightarrow (ox,oy) = (+0.15, -0.15)$.
 - **Visibility Effect:** Pushing vertices positively along x and negatively along y based on their depth (z) exposes the **bottom face** and **right face** of the geometries in 2.5D space.
2. **Assigned 3D Solid (Last Digit = 1):**
 - **Assigned Shape: Triangular Prism.**
 - **Vertex Allocation:**
 - **Cube (First Object):** $6 \text{ faces} \times 2 \text{ triangles/face} \times 3 \text{ vertices} = \mathbf{36 \text{ vertices}}$.
 - **Triangular Prism (Second Object):** $2 \text{ triangular bases} + 3 \text{ rectangular side faces} = \mathbf{24 \text{ vertices}}$.
 - **Total Buffer Vertices:** $36 + 24 = 60 \text{ vertices}$.

2. Technical Implementation Details

A. Shader Pipeline & Data Attributes

- **Vertex Shader (vsSource):** Accepts 3D coordinates (aPosition, size = 3) and RGBA color vectors (aVertexColor, size = 4). Passes interpolated colors to the fragment shader via vColor (varying).
- **Depth Projection Function (applyDepthOffset):** Modifies vertex positions in the array before binding to the WebGL position buffer using:
$$x_{draw} = x + ox \cdot (z + 0.5)$$
$$y_{draw} = y + oy \cdot (z + 0.5)$$

This projects relative depth without full 4x4 matrix transformation operations.

B. Color Schemes & Interpolation

- **Cube:** Features a smooth multi-color interpolated gradient across its front face and distinct solid flat colors on the remaining 5 faces.

- **Triangular Prism:** Features an interpolated RGB gradient across the front triangular base and solid colors across the back base and sloped side faces.

C. Depth Buffer & Render States

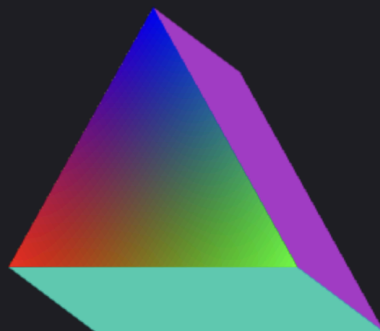
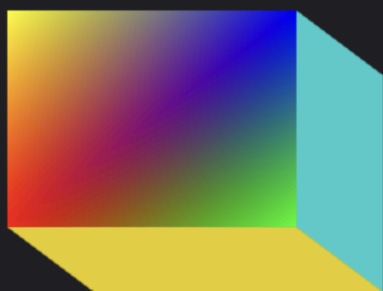
- Depth testing is managed dynamically using `gl.enable(gl.DEPTH_TEST)` and `gl.depthFunc(gl.LEQUAL)`.
- Dual rendering passes are configured to demonstrate draw-order dependence when depth testing is disabled (`gl.disable(gl.DEPTH_TEST)`).

3. Interactive Keyboard Navigation

Key Press	WebGL Primitive Mode / Render State	Description
1	<code>gl.TRIANGLES</code>	Renders filled solid polygons.
2	<code>gl.LINE_LOOP</code>	Renders continuous wireframe loops.
3	<code>gl.LINES</code>	Renders individual disconnected line segments.
4	<code>gl.LINE_STRIP</code>	Renders connected line strips.
5	<code>gl.POINTS</code>	Renders distinct vertex points (<code>gl_PointSize = 8.0</code>).
6	<code>gl.TRIANGLE_STRIP</code>	Renders connected triangle strips.
D	Toggle Depth Testing	Toggles <code>gl.DEPTH_TEST</code> ON or OFF.
S	Swap Draw Order	Reverses the rendering order (Cube → Prism vs. Prism → Cube).

4. Visual Results & Screenshots

- Default View (`gl.TRIANGLES`, Depth: ON):**
 - Shows the Cube on the left and Triangular Prism on the right.
 - Demonstrates correct 2.5D projection exposing the bottom and right faces.
- Wireframe Mode (`gl.LINE_LOOP` or `gl.POINTS`):**
 - Demonstrates primitive mode switching via keyboard control.
- Depth Buffer Disabled (`gl.DEPTH_TEST: OFF`):**
 - Highlights overlapping visual artifacts depending on the active draw order (S key).



ID: 21 | Mode: TRIANGLES | Depth: ON | Order: Cube First